Universidad de Granada

Periféricos y Dispositivos de Interfaz Humana
Curso 2025/2026

Práctica 2. Uso de bibliotecas de programación de interfaces de usuario en modo texto

Autora:
Inés Ruiz Sánchez

3 de abril de 2026

UNIVERSIDAD
DE GRANADA

Índice

1. Introducción **2**

2. Requisitos mínimos **2**
2.1. Instalación . . . 2
2.2. Ej1 - Hola mundo . . . 3
2.3. Ej2 - Ventana . . . 4
2.4. Ej3 - Pelota . . . 7
2.5. Juego sencillo . . . 8

3. Requisitos ampliados **10**
3.1. Ejercicio 4 - Pantalla de inicio . . . 10
3.2. Ejercicio 5 - Pantalla resumen . . . 10

4. Anexo **10**
4.1. Ejemplo completo para *ej1* . . . 10
4.2. Ejemplo completo para *ej2* . . . 11
4.3. Ejemplo modificado de *ej2* . . . 11
4.4. Ejemplo completo para *ej3* . . . 12
4.5. Código completo para el ejercicio 4 . . . 13
4.6. Código completo para el ejercicio 5 . . . 15

Índice de figuras

1. Comandos de instalación . . . 2
2. Continuar instalación . . . 3
3. Ejecución de hello.c . . . 4
4. Fichero ventana.c y su compilación . . . 5
5. Ejecución de ventana.c . . . 6
6. Ejecución de ventana.c con modificación . . . 7
7. Ejecución de pelotita.c . . . 8
8. Ejecución de pong.c . . . 10

1. Introducción

"ncurses" es una biblioteca de programación que provee una API que permite al programador escribir interfaces basadas en texto.

Los objetivos concretos de esta práctica son:

- instalar la librería *ncurses* en Linux
- crear programas sencillos basados en *ncurses*

Nota: Las tildes en los comentarios de los ficheros importados con extensión *.c* no se reconocen en LATEX por lo que el carácter no se escribe. En el *Anexo* se han importado los ficheros al completo y es por eso por lo que o bien no aparece un caracter o bien el caracter no tiene tilde, pareciendo una falta ortográfica.

2. Requisitos mínimos

Como requisitos mínimos se deben realizar y documentar adecuadamente las siguientes tareas:

- Instalar la librería ncurses, crear los programas de ejemplo ofrecidos (ej1, ej2 y ej3), y comprobar su funcionamiento.
- Crear un juego sencillo tipo "pong" partiendo del ejemplo del movimiento de la pelotita.

2.1. Instalación

Esta práctica se realiza en una máquina virtual con Ubuntu 22.
Instalar la librería necesaria:

```
sudo apt-get update
sudo apt-get install libncurses5-dev libncursesw5-dev
```

Listing 1: Comandos para instalar la librería

A continuación se muestra una imagen usando el segundo comando[1]:

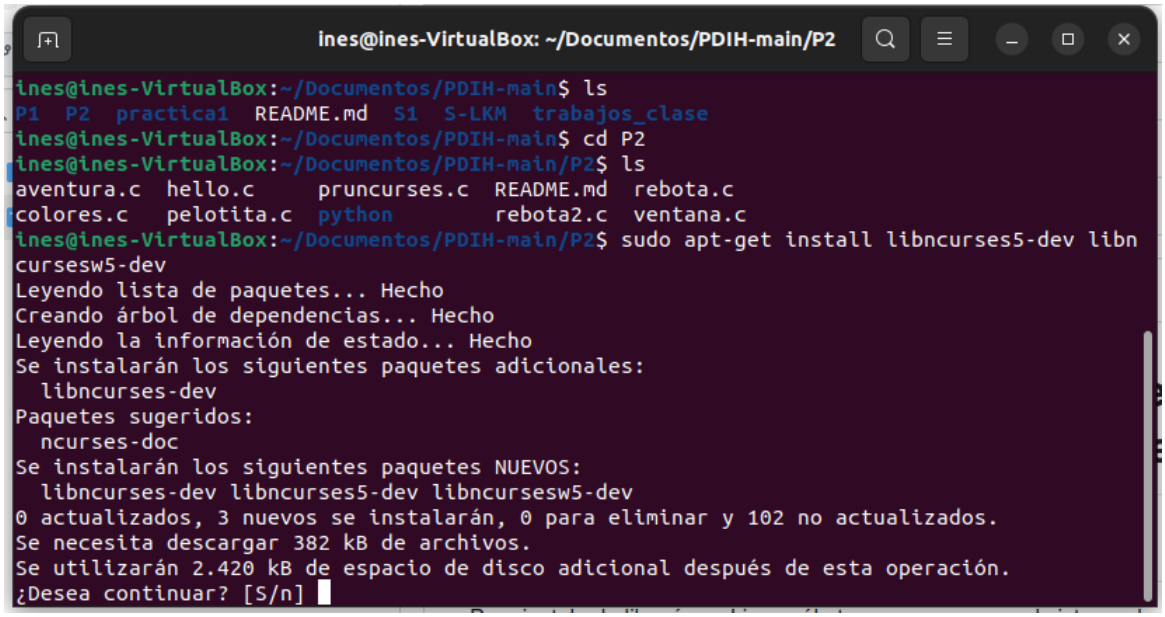

Figura 1: Comandos de instalación

Pulsamos la tecla S para continuar.

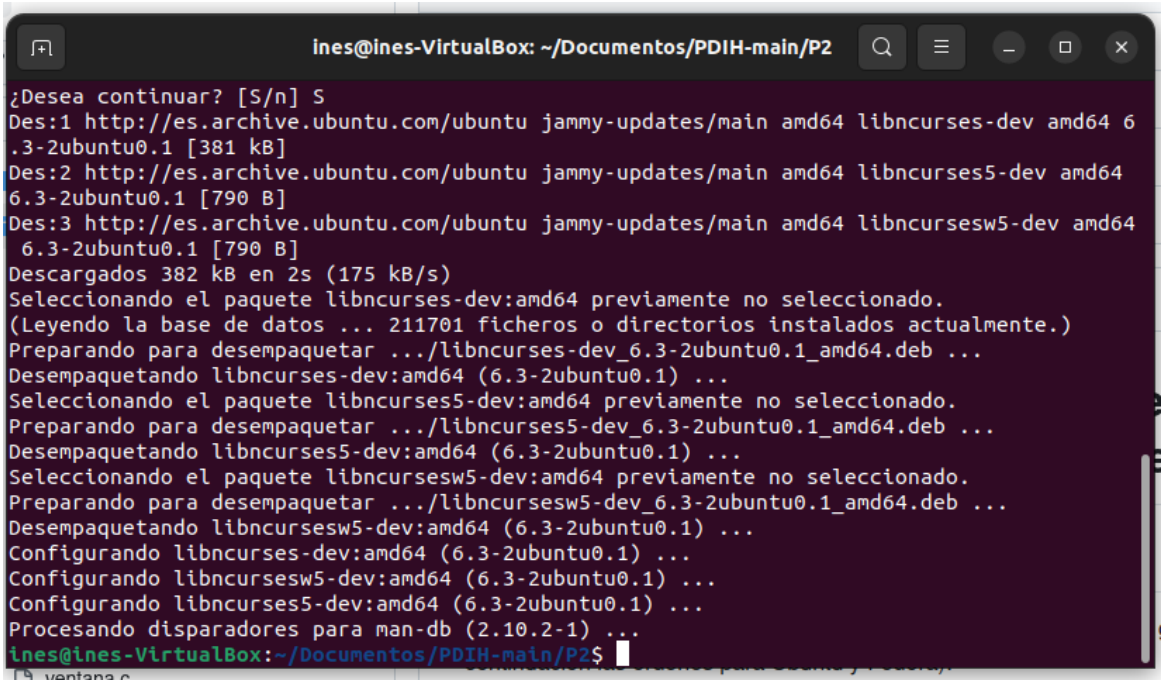

Figura 2: Continuar instalación

La instalación ha terminado con exito.

2.2. Ej1 - Hola mundo

Ejemplo proporcionado por el profesor:

```c
#include <ncurses.h>
#include <stdio.h>
int main() {
    initscr(); // Reserva memoria para la ventana principal
    printw("Hello␣World!"); // Lo escribe en la memoria
    refresh(); // Pasa lo que hay en memoria a la pantalla para que se vea
    getch();
    endwin(); // Cierra el modo ncurses
    return 0;
}
```

Listing 2: Fichero hello.c

Orden para compilar:

```
gcc hello.c -o hello -lncurses
```

Listing 3: Orden para compilar ej1

Al ejecutar el programa[2] se verá:

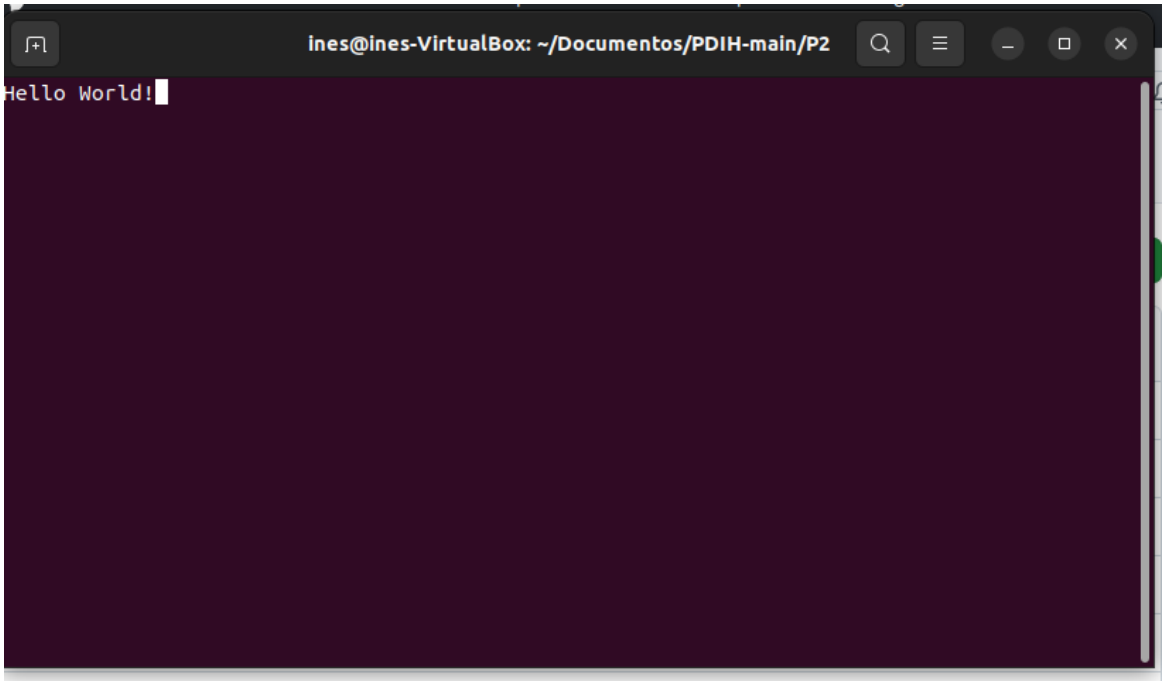

Figura 3: Ejecución de hello.c

El código de este ejercicio se encuentra en el anexo 4.1.

2.3. Ej2 - Ventana

Se trata del fichero *ventana.c*, incluido en el anexo 4.2

```c
int main(void) {
    int rows, cols;

    initscr();

    if (has_colors() == FALSE) { // Comprobar soporte de color
        endwin();
        printf("Your terminal does not support color\n");
        exit(1);
    }

    start_color(); // Activar soporte color

    // Crear combinaci n de colores
    init_pair(1, COLOR_YELLOW, COLOR_GREEN); // Texto amarillo, fondo verde
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3,COLOR_WHITE,COLOR_BLUE);
    clear();

    refresh();
    getmaxyx(stdscr, rows, cols);

    WINDOW *window = newwin(rows,cols,0,0); // Crea una ventana con las dimensiones de la terminal
    wbkgd(window, COLOR_PAIR(3));
    box(window, '|', '-'); // Dibuja el marco de la ventana (WINDOW, caracter vertical, caracter horizontal)

    mvwprintw(window, 10, 10, "una cadena");
    wrefresh(window);

    getch();
```

```
    endwin();
    return 0;
}
```

Listing 4: Programa ventana.c

Compilar con la orden:

```
    gcc ventana.c -o ventana -lncurses
```

Listing 5: Compilación de ventana.c

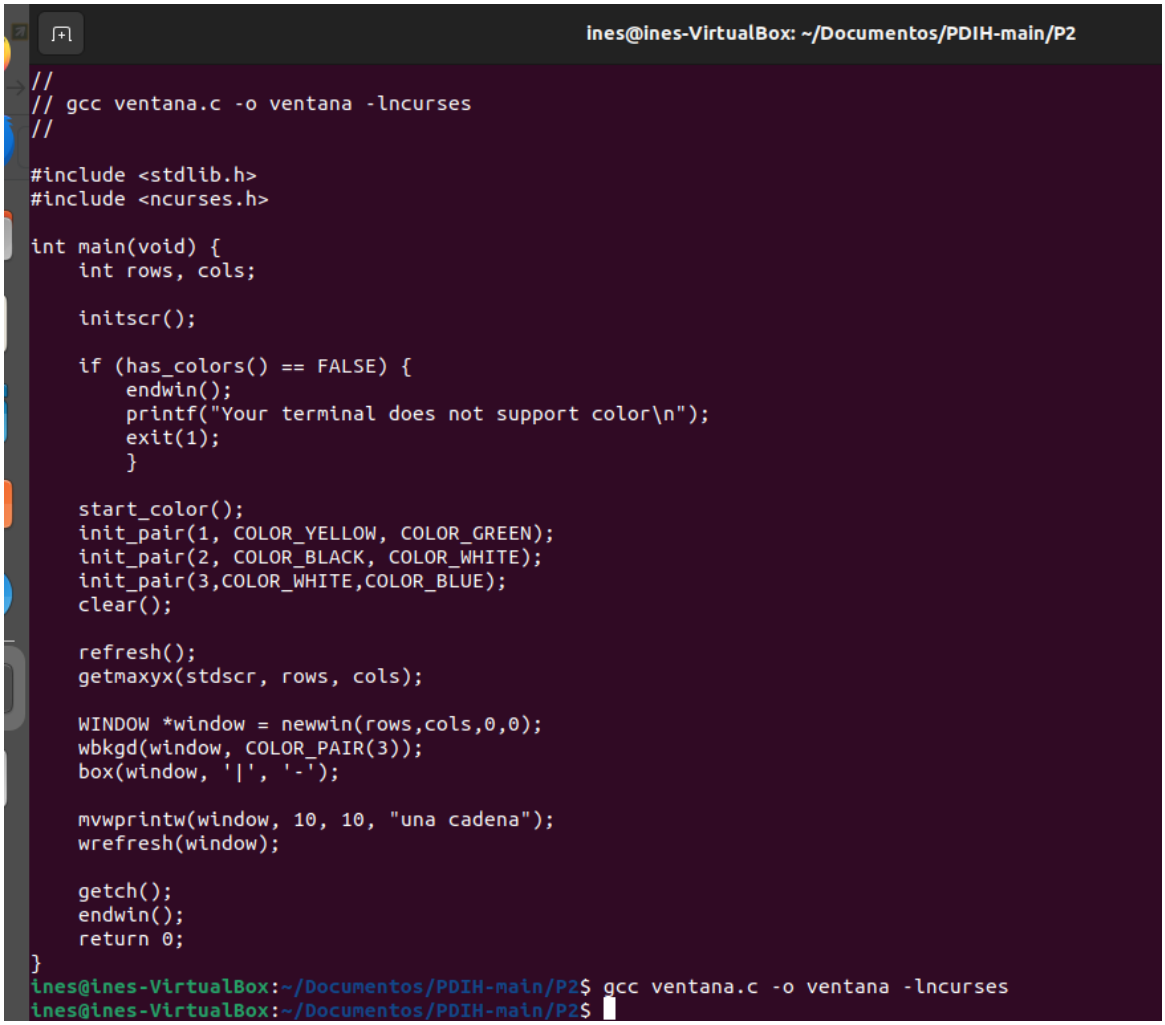

Figura 4: Fichero ventana.c y su compilación

Ejecución del programa *ventana*:

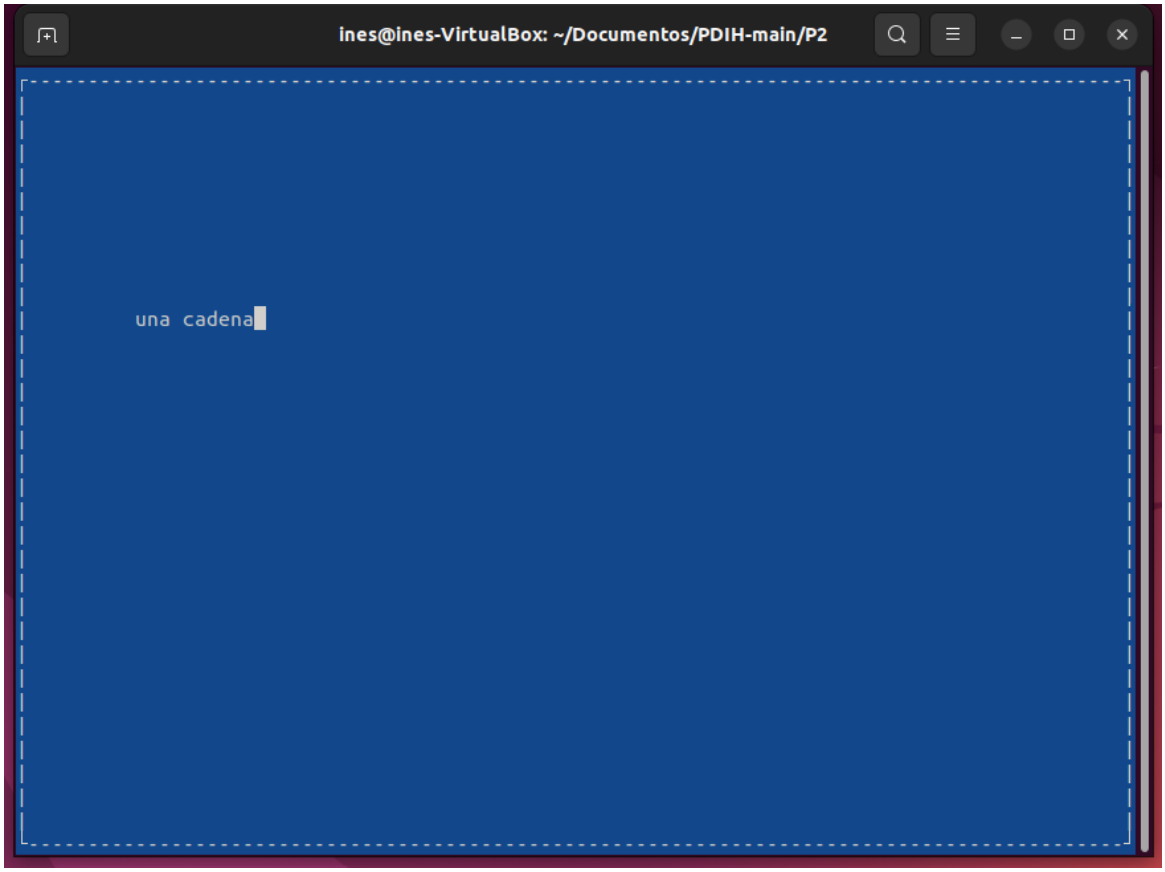

Figura 5: Ejecución de ventana.c

Una modificación de ese ejemplo se puede ver en el anexo 4.2. Las líneas que se han cambiado con respecto al fichero inicial son:

```
//getmaxyx(stdscr, rows, cols);
rows=10;
cols=40;

WINDOW *window = newwin(rows,cols,5,10); // Modificaci n del tama o
wbkgd(window, COLOR_PAIR(1)); // Cambio de colores: amarillo y verde
box(window, '*', '*'); // Modificaci n de marco
```

Listing 6: Modificación de ventana.c

Ejecución:

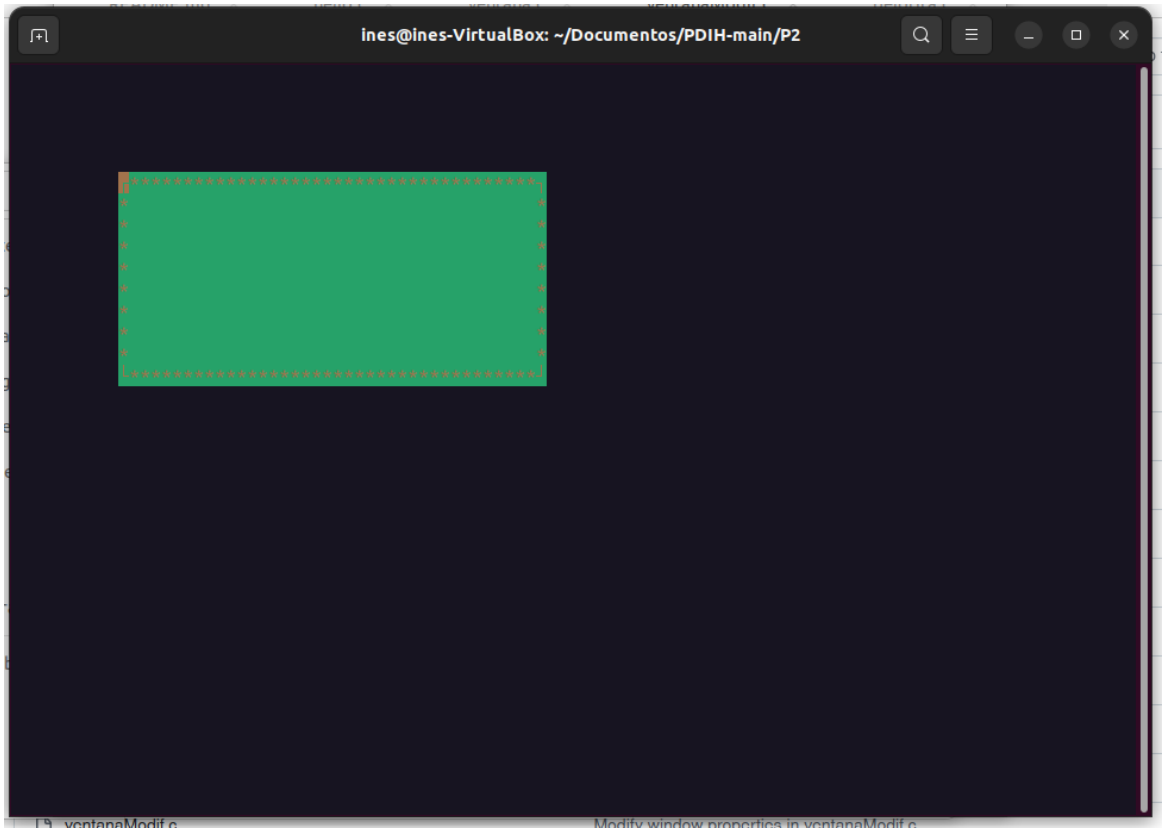

Figura 6: Ejecución de ventana.c con modificación

Se pueden observar los cambios:

- con la función *box* se cambian los caracteres para el marco: *
- con la función *wbkgd* se cambia el color de la ventana: verde para el fondo y amarillo para los caracteres
- con la función *newwin* se modifica el tamaño de la ventana indicando alto, ancho, inicio de y, inicio de x

2.4. Ej3 - Pelota

Fichero *pelotita.c* proporcionado por el profesor 4.4:

```c
#include <ncurses.h>
#include <unistd.h>

#define DELAY 30000

int main(int argc, char *argv[]) {
 int x = 0, y = 0;
 int max_y = 20, max_x = 20;
 int next_x = 0;
 int direction = 1;

 initscr();
 noecho(); // Para no mostrar por pantalla las teclas que se pulsan
 curs_set(FALSE); // Ocultar el cursor

 while(1) {
    clear();
    mvprintw(y, x, "o");
    refresh();

    usleep(DELAY); // Pausa para que nos de tiempo a ver la pelota
```

```c

    next_x = x + direction;

    if (next_x >= max_x || next_x < 0) {
        direction*= -1;
    } else {
        x+= direction;
    }
 }

 endwin();
}
```

Listing 7: Programa pelotita.c

Para compilar hay que usar la siguiente orden:

```c
    gcc pelotita.c -o pelotita -lncurses
```

Listing 8: Compilacion pelotita.c

Ejecución:

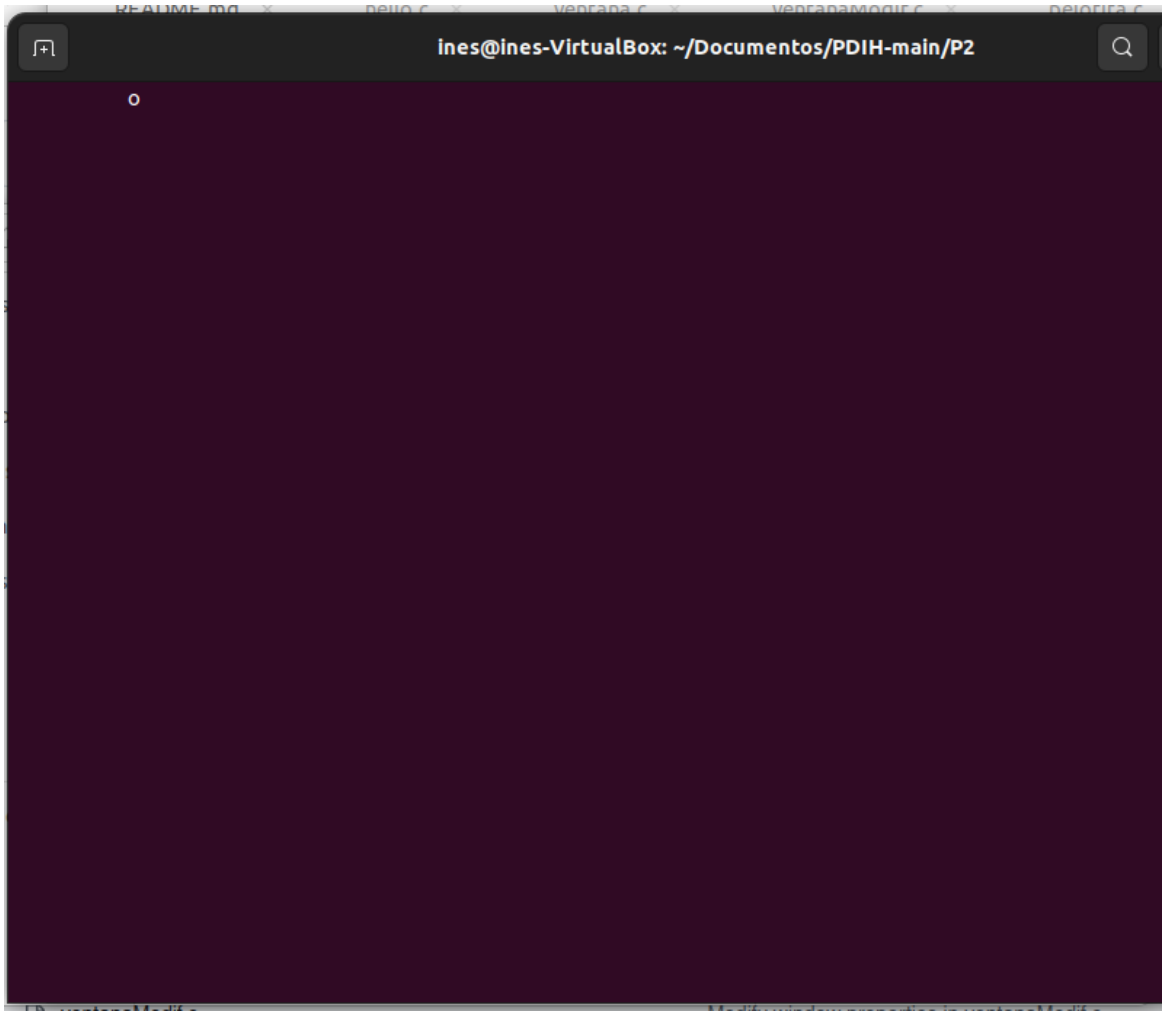

Figura 7: Ejecución de pelotita.c

2.5. Juego sencillo

Para crear el juego partimos del fichero pelotita.c (anexo 4.4), pero hay que añadirle las palas y la lógica de la puntuación. Los cambios han sido:

- Añadir movimiento en el Eje Y para la pelota.

```
        int pelota_x = max_x /2;
        int pelota_y = max_y /2;
        int dir_x = 1;
        int dir_y = 1;
        ...
        pelota_x += dir_x ;
        pelota_y += dir_y ;
```

Listing 9: Movimiento de pelota

- Añadir las dos palas, solo con movimiento en Eje Y.

```
        int pala_izq_y = ( max_y /2) - ( LONGITUD_PALA /2) ;
        int pala_der_y = ( max_x /2) - ( LONGITUD_PALA /2) ;
```

Listing 10: Movimiento de palas

- Capturar las teclas de juego con la función *getch()*

```
        int tecla = getch();
        if (tecla == 'w' || tecla == 'W')
            if (pala_izq_y > 0) pala_izq_y--;
        if (tecla == 's' || tecla == 'S')
            if(pala_izq_y < max_y - LONGITUD_PALA) pala_izq_y++;
        if (tecla == KEY_UP)
            if (pala_der_y > 0) pala_der_y--;
        if (tecla == KEY_DOWN)
            if (pala_der_y < max_y - LONGITUD_PALA) pala_der_y++;
```

Listing 11: Captación de teclas

  - El jugador 1 usa las teclas W y S para mover su pala.
  - El jugador 2 usa las teclas con **flechas** para mover su pala.

- Marcador de puntos

```
        int puntos_j1 = 0;
        int puntos_j2 = 0;
        ...
        if ( pelota_x <= 0) { // Punto para el jugador 2
            puntos_j2 ++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = 1;
        } else if ( pelota_x >= max_x - 1) { // Punto para el jugador 1
            puntos_j1 ++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = -1;
        }
```

Listing 12: Puntuación

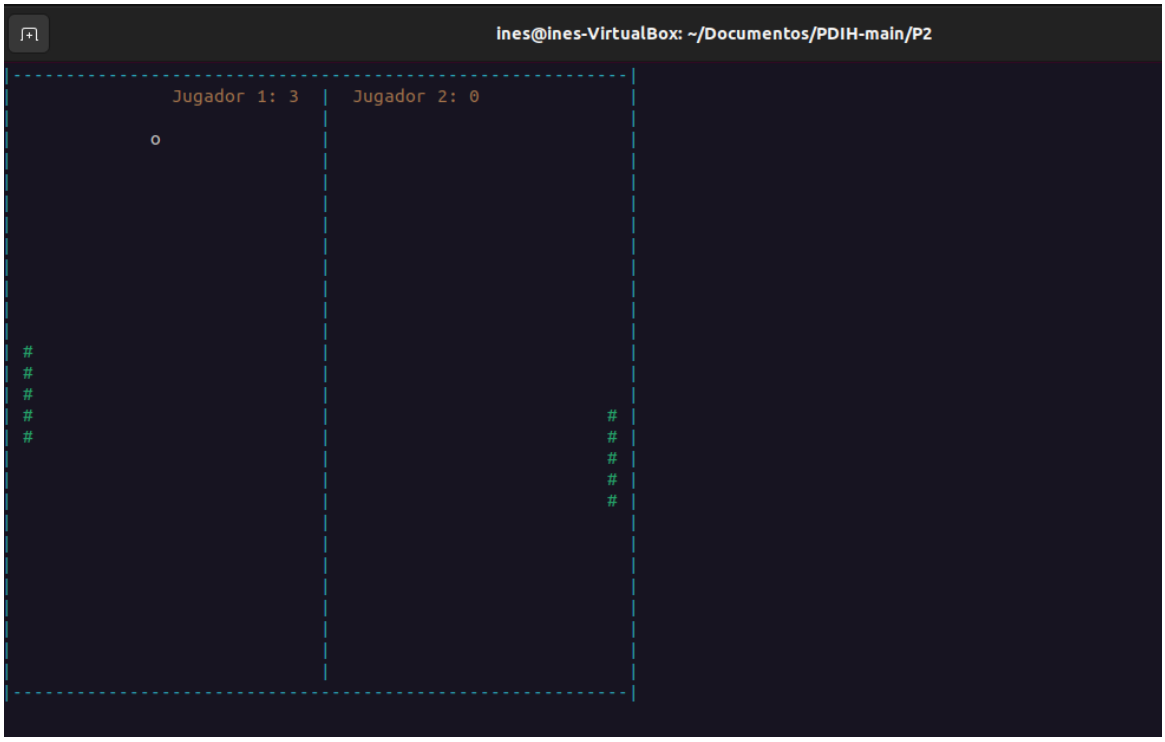

Figura 8: Ejecución de pong.c

3. Requisitos ampliados

Como requisitos ampliados (opcionales para subir nota) se propone:

- que al iniciar el juego se muestre una **pantalla de bienvenida** en la que se muestren los datos de quienes han realizado el juego y explicando los controles de juego (p.ej. un recuadro con la explicación). Tras una pausa o pulsación de tecla se iniciará el juego en sí mismo.
- que al terminar cada partida se muestre una **pantalla de resumen** mostrando el marcador final y felicitando al ganador. Se dará la opción de volver a jugar o terminar el programa.

3.1. Ejercicio 4 - Pantalla de inicio

En el juego del *pong*, al iniciar el juego se mostrará una pantalla de bienvenida en la que se muestren los datos de quienes han realizado el juego y explicando los controles de juego (p.ej. un recuadro con la explicación). Tras una pausa o pulsación de tecla se iniciará el juego en sí mismo.

Para ver el código completo: anexo4.5.

3.2. Ejercicio 5 - Pantalla resumen

En el juego del *pong*, al terminar cada par da se mostrará una pantalla de resumen mostrando el marcador final y felicitando al ganador. Se dará la opción de volver a jugar o terminar el programa.

Para ver el código completo: anexo4.6.

4. Anexo

4.1. Ejemplo completo para *ej1*

Código proporcionado por el profesor:

```c
#include <ncurses.h>
#include <stdio.h>

int main() {
    initscr(); // Reserva memoria para la ventana principal
    printw("Hello World!"); // Lo escribe en la memoria
    refresh(); // Pasa lo que hay en memoria a la pantalla para que se vea
    getch();
    endwin(); // Cierra el modo ncurses
    return 0;
}
```

4.2. Ejemplo completo para *ej2*

```c
#include <stdlib.h>
#include <ncurses.h>

int main(void) {
    int rows, cols;

    initscr();

    if (has_colors() == FALSE) { // Comprobar soporte de color
        endwin();
        printf("Your terminal does not support color\n");
        exit(1);
    }

    start_color(); // Activar soporte color

    // Crear combinaci n de colores
    init_pair(1, COLOR_YELLOW, COLOR_GREEN); // Texto amarillo, fondo verde
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3,COLOR_WHITE,COLOR_BLUE);
    clear();

    refresh();
    getmaxyx(stdscr, rows, cols);

    WINDOW *window = newwin(rows,cols,0,0); // Crea una ventana del tama o de la
        terminal
    wbkgd(window, COLOR_PAIR(3));
    box(window, '|', '-'); // Dibuja el marco de la ventana

    mvwprintw(window, 10, 10, "una cadena");
    wrefresh(window);

    getch();
    endwin();
    return 0;
}
```

4.3. Ejemplo modificado de *ej2*

```c
#include <stdlib.h>
#include <ncurses.h>

int main(void) {
    int rows, cols;

```

```
    initscr();

    if (has_colors() == FALSE) {
        endwin();
        printf("Your␣terminal␣does␣not␣support␣color\n");
        exit(1);
    }

    start_color();
    init_pair(1, COLOR_YELLOW, COLOR_GREEN);
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3,COLOR_WHITE,COLOR_BLUE);
    clear();

    refresh();
    //getmaxyx(stdscr, rows, cols);
    rows=10;
    cols=40;

    WINDOW *window = newwin(rows,cols,5,10); // Modificaci n del tama o
    wbkgd(window, COLOR_PAIR(1)); // Cambio de colores: amarillo y verde
    box(window, '*', '*'); // Modificaci n de marco

    mvwprintw(window, 10, 10, "una␣cadena");
    wrefresh(window);

    getch();
    endwin();
    return 0;
}
```

4.4. Ejemplo completo para *ej3*

```
#include <ncurses.h>
#include <unistd.h>

#define DELAY 30000

int main(int argc, char *argv[]) {
 int x = 0, y = 0;
 int max_y = 20, max_x = 20;
 int next_x = 0;
 int direction = 1;

 initscr();
 noecho(); // Para no mostrar por pantalla las teclas que se pulsan
 curs_set(FALSE); // Ocultar el cursor

 while(1) {
    clear();
    mvprintw(y, x, "o");
    refresh();

    usleep(DELAY); // Pausa para que nos de tiempo a ver la pelota

    next_x = x + direction;

    if (next_x >= max_x || next_x < 0) {
        direction*= -1;
    } else {
        x+= direction;
    }
```

```
 }

 endwin();
}
```

4.5. Código completo para el ejercicio 4

```
#include <ncurses.h>
#include <unistd.h>
#include <stdlib.h>

#define DELAY 50000 // Velocidad de la pelota
#define LONGITUD_PALA 5 // Altura de las palas

int main(int argc, char *argv[]) {
 // Dimensiones del tablero
 int max_y=30, max_x=60;

 // Inicializar entorno de ncurses
 initscr();
 start_color(); // Activar soporte de color
 cbreak();
 noecho(); // Para no mostrar por pantalla las teclas que se pulsan
 kpad(stdscr, TRUE);
 curs_set(FALSE); // Ocultar el cursor
 nodelay(stdscr, TRUE);

 // Configurar los colores
 init_pair(1, COLOR_WHITE, COLOR_BLACK);
 init_pair(2, COLOR_GREEN, COLOR_BLACK);
 init_pair(3, COLOR_YELLOW, COLOR_BLACK);
 init_pair(4, COLOR_CYAN, COLOR_BLACK);

 getmaxyx(stdscr, max_y, max_x); // Obtiene dimensiones dinamicamente

 // Variables del juego
 int pelota_x = max_x /2;
 int pelota_y = max_y /2;
 int dir_x = 1;
 int dir_y = 1;

 // Posicion de las palas
 int pala_izq_y = (max_y /2) - (LONGITUD_PALA /2);
 int pala_der_y = (max_x /2) - (LONGITUD_PALA /2);

 // Puntuacion
 int puntos_j1 = 0;
 int puntos_j2 = 0;

 while(1) {
    // Leer entrada de teclado
    int tecla = getch();

    // Jugador 1
    if(tecla == 'w' || tecla == 'W'){
        if(pala_izq_y > 1){
            pala_izq_y--;
        }
    }
    if(tecla == 's' || tecla == 'S'){
        if(pala_izq_y < max_y - LONGITUD_PALA -1){
            pala_izq_y++;
```

```
        }
    }

    // Jugador 2
    if(tecla == KEY_UP){
        if(pala_derq_y > 1){
            pala_der_y--;
        }
    }
    if(tecla == KEY_DOWN){
        if(pala_der_y < max_y - LONGITUD_PALA -1){
            pala_der_y++;
        }
    }

    // Salir del juego
    if(tecla == 'q' || tecla == 'Q'){
        break;
    }

    // Actualizar posicion de la pelota
    pelota_x += dir_x;
    pelota_y += dir_y;

    // Colisiones
    // Rebote en techo y suelo
    if(pelota_y <=1 || pelota_y >= max_y -2){
        dir_y *= -1;
    }

    // Rebote en la pala izquierda
    if (pelota_x == 3 && pelota_y >= pala_izq_y && pelota_y < pala_izq_y + LONGITUD_PALA) {
        dir_x *= -1;
    }

    // Rebote en la pala derecha
    else if (pelota_x == max_x - 4 && pelota_y >= pala_der_y && pelota_y < pala_der_y + LONGITUD_PALA) {
        dir_x *= -1;
    }

    // Puntuacion
    if (pelota_x <= 0) { // Punto para el jugador 2
            puntos_j2++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = 1;
    } else if (pelota_x >= max_x - 1) { // Punto para el jugador 1
            puntos_j1++;
            pelota_x = max_x / 2; pelota_y = max_y / 2;
            dir_x = -1;
    }

    // Dibujar los elementos en pantalla
        clear();

        // Marco
        attron(COLOR_PAIR(4));
        for(int i = 0; i < max_x; i++) {
            mvprintw(0, i, "-");          // Techo
            mvprintw(max_y - 1, i, "-");  // Suelo
        }
        for(int i = 0; i < max_y; i++) { // Paredes
```

```
            mvprintw(i, 0, "|");           // Pared izquierda
            mvprintw(i, max_x - 1, "|");  // Pared derecha
        }
        for(int i = 1; i < max_y - 1; i++) {// La red del centro
            mvprintw(i, max_x/2, "|");
        }
        attroff(COLOR_PAIR(4));

        // Marcador
        attron(COLOR_PAIR(3));
        mvprintw(1, max_x/2 - 15, "␣Jugador␣1:␣%d␣", puntos_j1);
        mvprintw(1, max_x/2 + 2, "␣Jugador␣2:␣%d␣", puntos_j2);
        attroff(COLOR_PAIR(3));

        // Palas
        attron(COLOR_PAIR(2));
        for(int i = 0; i < LONGITUD_PALA; i++) {
            mvprintw(pala_izq_y + i, 2, "#");          // Pala izquierda
            mvprintw(pala_der_y + i, max_x - 3, "#"); // Pala derecha
        }
        attroff(COLOR_PAIR(2));

        // Pelota
        attron(COLOR_PAIR(1));
        mvprintw(pelota_y, pelota_x, "o");
        attroff(COLOR_PAIR(1));
    refresh();

    usleep(DELAY); // Pausa para que nos de tiempo a ver la pelota

 }

 // Finalizar
 endwin();
 return 0;
}
```

4.6. Código completo para el ejercicio 5